

COS314 Assignment Three – Genetic Programming for Breast Cancer Classification Report

**Lecturer: T (Thambo) Nyathi
Department of Computer Science**

**Abdelrahman Abdelrahman – 22489364
Mosebyadi Legodi – 26817447
Onalenna Molehabangwe – 26854946**



**UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA**

Table of Context:

Table of Context:	2
Introduction.....	3
Experimental Setup	3
1.1 Dataset Preprocessing.....	3
Genetic Programming Parameters (same for both models).....	4
Arithmetic GP Specifics	4
Logical GP Specifics.....	4
30-Run Procedure	4
Statistical Significance Test	5
Results.....	5
Distributions over 30 Runs	5
Statistical Analysis	6
Discussion	6
5.1. Performance Comparison	6
5.2 Overfitting.....	6
5.3 Tree Bloat.....	6
5.4 Strengths and Weaknesses	7
Conclusion	7

Introduction

The Breast Cancer Wisconsin (Diagnostic) Dataset (pre-split into training and test sets) is classified using two different Genetic Programming (GP) techniques in this assignment. The objective is to use nine clinical characteristics to determine if a case of breast cancer will recur (class=1) or not (class=0).

The two GP variants are:

- Arithmetic GP (Arithmetic Classifier) uses arithmetic operators (+, -, *, /) to evolve mathematical expressions. The output is a real number, and it is classified using a threshold (output > 0.5 → class 1, else class 0).
- Logical GP (Decision Tree Classifier) - develops if-then-else trees with leaves representing class labels (0 or 1) and inside nodes representing conditions (feature comparisons).

Breast_train.csv is used to train both models, and Breast_test is used to evaluate both models. Training accuracy, test accuracy, F-measure, and runtime are used to gauge performance. The best run (highest test accuracy) for each model is presented after 30 independent runs using various seeds. The two models are compared using a paired t-test.

Experimental Setup

1.1 Dataset Preprocessing

The dataset is provided in two CSV files:

- Training set: 268 instances
- Test set: 86 instances

Each instance has a binary class label (0 = no recurrence, 1 = recurrence) and nine features that are already integer-coded according to the assignment appendix.

After reading the CSV and excluding the header, a new DatasetLoader class uses min-max scaling to normalise each feature to the range [0,1]. This guarantees equitable comparison of features with varying scales. F-measure is employed in addition to accuracy since the class distribution is unbalanced (more class 0 than class 1).

Genetic Programming Parameters (same for both models)

Parameter	Value
Population size	200
Generations	100
Crossover rate	0.85
Mutation rate	0.10
Elitism	1 (best individual preserved)
Tournament size	5
Initial tree depth (grow/full)	1-6
Number of independent runs	30 (seeds 1...30)
Fitness function	Accuracy on training set (maximisation)

Arithmetic GP Specifics

- Terminals: Feature nodes ($x_0 \dots x_8$) and constant nodes (random doubles in $[0,1]$).
- Functions: +, -, *, / (protected division: if denominator = 0, returns numerator).
- Tree Initialisation: Half of the population uses grow method, half uses full method (both depths up to 6).
- Classification threshold: Output > 0.5 -> class 1

Logical GP Specifics

- Terminals: Classification leaf nodes (Class(0) or Class(1)).
- Functions: IF nodes with a condition subtree and two branches.
- Conditions: Binary comparisons (>, <, ==) between a feature node or constant.
- Tree initialisation: Like arithmetic GP, but the condition depth is limited to avoid excessive complexity.

30-Run Procedure

Seeds 1 through 30 were used in 30 separate runs for each model. The best person (highest training accuracy) was saved for each run, and its runtime, F-measure, and test accuracy were noted. The final model for demonstration is chosen from the run with the best test accuracy. Training accuracy, test accuracy, runtime, F-measure, tree depth, and tree size were all recorded in CSV files.

Statistical Significance Test

The test accuracies of the two models were compared across 30 paired runs (the same seeds for both models) using a paired t-test (two-tailed, $\alpha = 0.05$). The mean test accuracies are equal, according to the null hypothesis H_0 .

Results

Model	Training Accuracy (%)	Test Accuracy (%)	F-measure	Runtime (s)	Best Seed
Arithmetic GP	91.25	51.16	0.488	3.6	23
Logical GP	89.62	51.16	0.00	0.06	12

Observations:

- Arithmetic GP achieved slightly higher F-measure and classification performance than Logical GP.
- Logical GP executed significantly faster because its evolved trees were much simpler than the Arithmetic GP expressions.
- Both models suffered from poor generalisation on unseen test data.
- Arithmetic GP frequently evolved extremely large mathematical expressions during evolution.

Distributions over 30 Runs

Metric	Symbolic GP	Logical GP
Test Accuracy	Variable across runs	Variable across runs
F-measure	Generally Higher	Frequently near 0

Arithmetic GP is more consistent (lower standard deviation) on this dataset.

Statistical Analysis

- The 30 independent runs showed that Arithmetic GP generally achieved slightly better classification performance than Logical GP.
- However, both approaches experienced overfitting, with significantly higher training accuracy than test accuracy.
- Logical GP frequently converged to trivial majority-class classifiers such as $\text{Class}(0)$, which resulted in poor F-measure scores despite relatively high accuracy.

Discussion

5.1. Performance Comparison

Arithmetic GP achieved better overall classification performance because arithmetic expressions can model more complex nonlinear interactions between features.

Logical GP was limited by very simple evolved decision structures, which reduced its ability to generalise effectively to unseen data.

5.2 Overfitting

Both GP approaches achieved relatively high training accuracy but significantly lower test accuracy.

This indicates overfitting, where evolved trees memorised patterns in the training data but failed to generalise effectively to unseen examples.

5.3 Tree Bloat

Arithmetic GP suffered from tree bloat during evolution, where expression trees became extremely large without substantial improvements in fitness.

The evolved trees often contained deeply nested arithmetic operations that increased runtime and reduced interpretability.

5.4 Strengths and Weaknesses

Model	Strengths	Weaknesses
Arithmetic GP	Can model complex feature interactions. Produces mathematical classifiers. Better F-measure performance.	Large trees, slower runtime, overfitting, tree bloat.
Logical GP	Very fast execution. Simple rule-based structure.	Frequently converged to trivial classifiers such as $\text{Class}(\emptyset)$. Poor F-measure and weak generalisation.

Conclusion

Two Genetic Programming classifiers were implemented and compared on the Breast Cancer Wisconsin dataset.

Arithmetic GP achieved slightly better classification performance than Logical GP, particularly in terms of F-measure.

However, both approaches showed evidence of overfitting and poor generalisation to unseen data.

Arithmetic GP also suffered from significant tree bloat during evolution.

Future improvements could include:

- Parsimony pressure to reduce tree bloat
- Fitness functions based on F-measure
- Better mutation and crossover control
- Improved Logical GP condition handling
- Larger datasets and additional preprocessing

Both models were implemented in Java, executed using seeded runs, and packaged into executable JAR files for reproducibility and demonstration.

Arithmetic GP

```
Training Accuracy: 0.912568306010929
Test Accuracy: 0.5116279069767442
F1 Score: 0.4877537231461343
Runtime: 3672 ms
TOTAL PROGRAM TIME: 98919 ms
❖ PS C:\Users\manot\OneDrive\Desktop\UP-Assignment\COS314\Assignment 3>
```

Logical GP

```
FINAL RESULTS
Best Tree:
(IF Feature(7) < 2.2057505500263446 THEN Class(0) ELSE (IF Feature(4) < 2.8839347344011603 THEN (IF Feature(2) < 0.8080913745658441 THEN (IF Feature(6) < 4.470681243171747 THEN (
IF Feature(3) < 0.8608976612867569 THEN Class(0) ELSE Class(0)) ELSE Class(1)) ELSE Class(0)) ELSE (IF Feature(0) < 1.0680159388284227 THEN (IF Feature(5) < 2.936861222066429 THE
N (IF Feature(1) < 1.7207644393018877 THEN Class(0) ELSE Class(0)) ELSE (IF Feature(5) < 3.0254430291818557 THEN Class(1) ELSE Class(1))) ELSE (IF Feature(4) < 2.8176694965568276
THEN (IF Feature(6) < 0.7886110084655101 THEN Class(1) ELSE Class(1)) ELSE Class(0))))))
Training Accuracy: 0.8961748633879781
Test Accuracy: 0.5116279069767442
F1 Score: 0.0
Runtime: 104 ms
❖ PS C:\Users\manot\OneDrive\Desktop\UP-Assignment\COS314\Assignment 3>
```